

SageMaker Pipelines

Instructor

Kartik Nighania

MLOps Engineer at Typewise

Ex-Head of Engineering - YC-backed startup

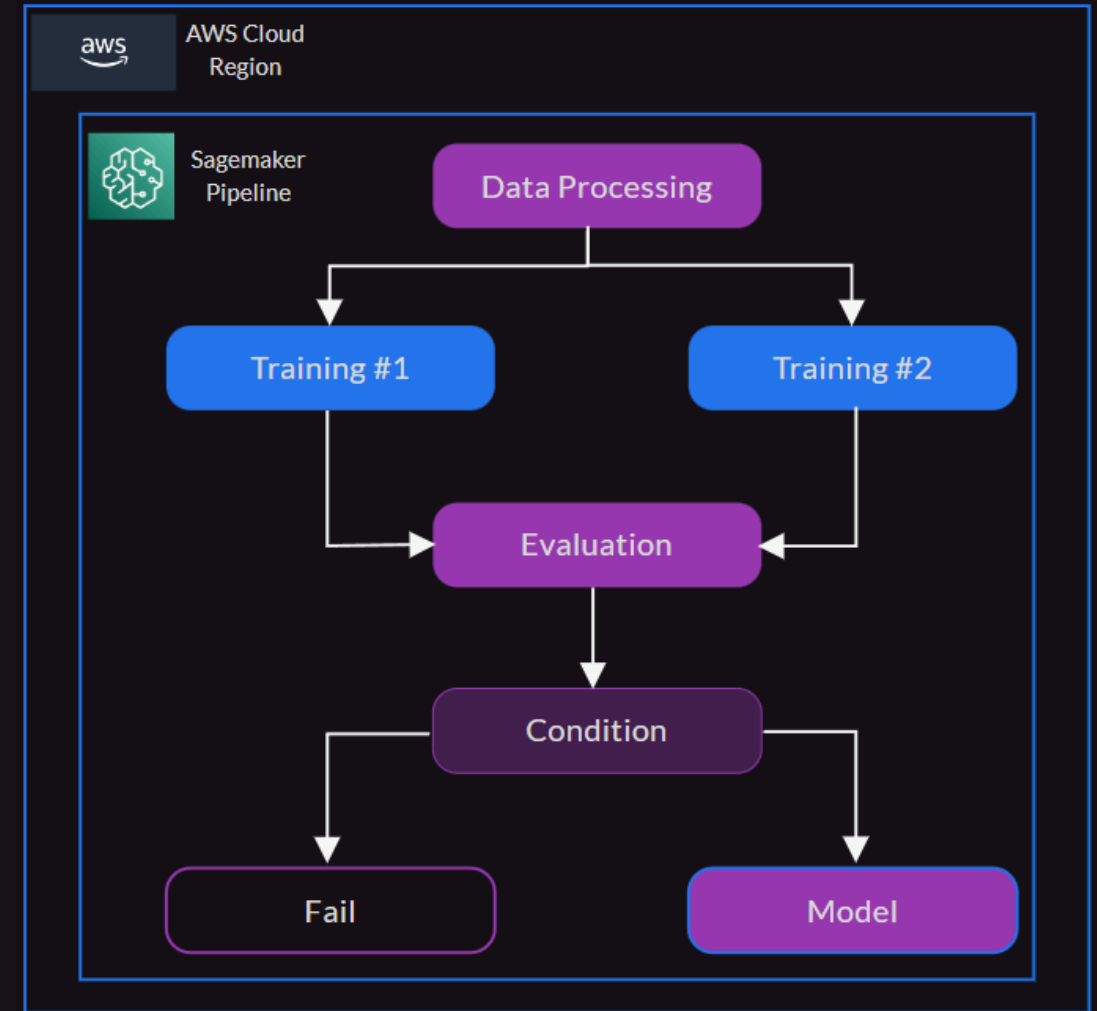
Certified AWS Cloud and Kubernetes Engineer

Open source contributor



SageMaker Pipeline

- A series of steps with a specific purpose
- A directed acyclic graph (DAG) is automatically created
- Automatically creates and manages EC2 infrastructure
- Available as a Python SDK
- Fully integrated with SageMaker Studio UI



Why do we need a Pipeline?



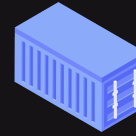
Automatic step retry
Step caching



Global parameters



Data Lineage Tracking
Integration with S3 for IO



Containerization



Infrastructure provisioning
Support for spot-instances



SageMaker Experiments integration
Automatically track hyperparameters
Model artifacts
Evaluation metadata
Other metrics

SageMaker Pipeline Steps

- **Processing Step**

- Used to create a processing job for data processing

- **Use case**

- For data pre and post processing
- For model evaluation after training
- Can also output files as evaluation reports

Processing

SageMaker Pipeline Steps

- How it works
 - Sagemaker starts a processing job with an EC2 machine
 - Takes the data from S3 and copies it over EC2
 - Runs the container with the given script
 - Copies the output artifacts back to S3
 - De-provisions the EC2 machine

Processing

Model Training Steps

- **Training Step:** Used to create a training job for training a model
- **Tuning Step:** Used to create a hyperparameter tuning job for a given model

Training

Tuning

Model Training Steps

Use Cases

- Train model with pre-built containers or use a custom container
- Hyperparameter tuning can run models in parallel to speed-up training time
- Based on the tuning objective top 50 performing versions are retained

Training

Tuning

Model Training Steps

How it works

- Sagemaker starts a job with EC2 machine(s)
- Takes the training and validation data from S3 and copies it over to EC2
- Runs the container(s) with the given script
- Tracks container logs
- Tracks metrics using the container logs regex pattern
- After training de-provisions the EC2 machine(s)

Training

Tuning

Model Inference Steps

Model Step

- Creates a new model for the model registry. The Model Registry is a versioned system of managing models and has all the information needed to successfully retrieve/deploy the versioned model

Model

Transform

Model Inference Steps

Transform Step

- Used to batch run inference of a model on a given dataset

Model

Transformers

Model Inference Steps

Transform Step

- Sagemaker starts a transform job with EC2 machine(s)
- Takes the data from S3 and optionally splits into chunks
- Runs inference on the dataset
- Optionally aggregates the data and store the output data to S3
- Afterwards de-provisions the EC2 machine(s)

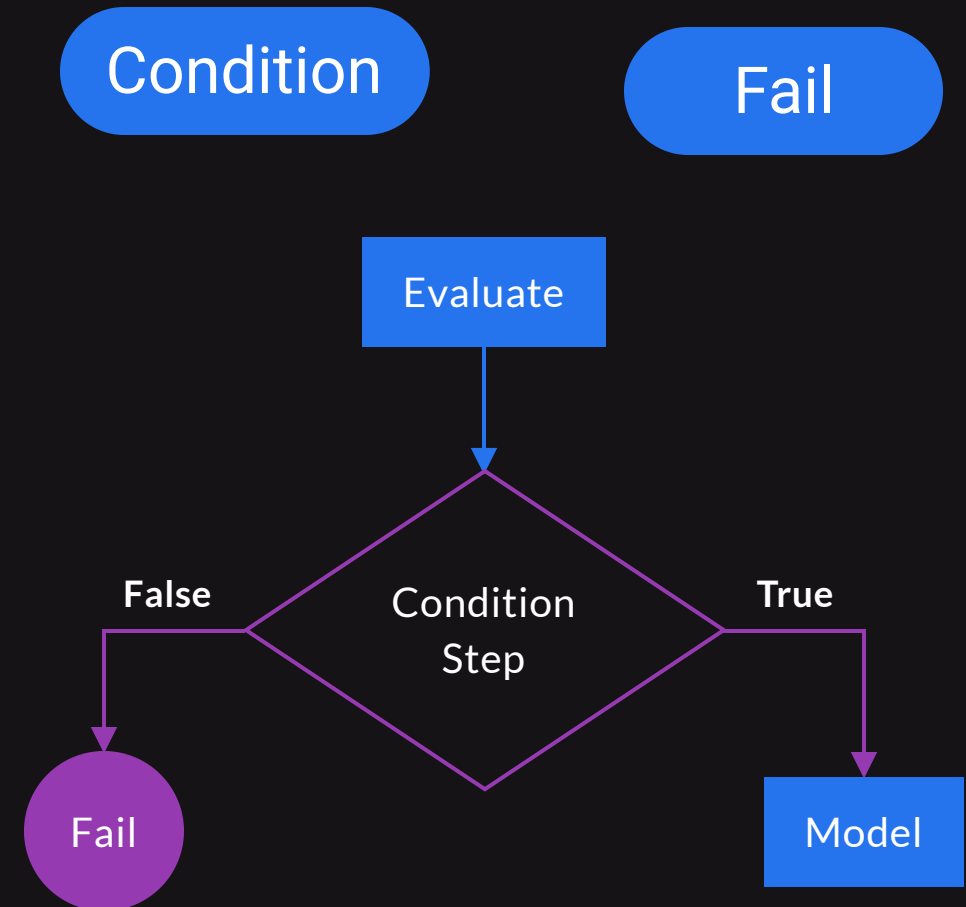
Model

Transformers

Logical Steps

Condition Step: compare step properties of a previous step with the defined condition(s) to take further steps based on evaluation results

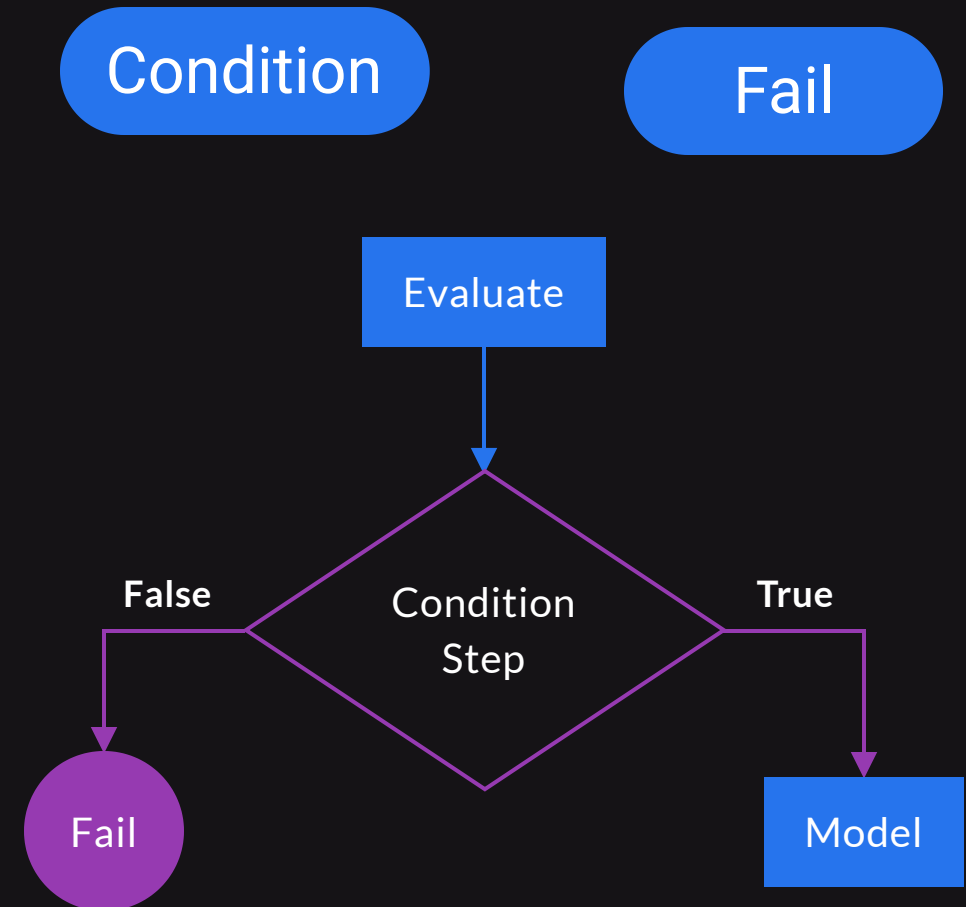
Fail step: stop execution when a desired condition or state is not achieved. This also marks that pipeline's execution as failed.



Logical Steps

Use Case:

- Fail the pipeline if a given target evaluation metric is not achieved
- Early stop the execution of a pipeline to save on AWS resources
- Do not register a given model if the performance is not acceptable



Event Based Systems

Callback

Lambda

Callback step: Send a message to AWS SQS queue and halt the pipeline till further response is received from the queue consumer



EventBridge

Event Based Systems

Callback

Lambda

Lambda step: Create a Lambda function or run an existing lambda function. The pipeline waits for the Lambda function (max 10 mins) or the execution fails.



EventBridge

Event Based Systems

AWS EventBridge: Service that can monitor status change events across various features in SageMaker. These triggers indicate changes in the state of SageMaker pipelines, jobs, models, etc. which can be used to further notify and automate systems.

Callback

Lambda



EventBridge

Event Based Systems

Use cases:

- Notify team on pipeline, job or training status
- Deploy a model when its state changes to approved
- Call other microservices and notify transform job completion

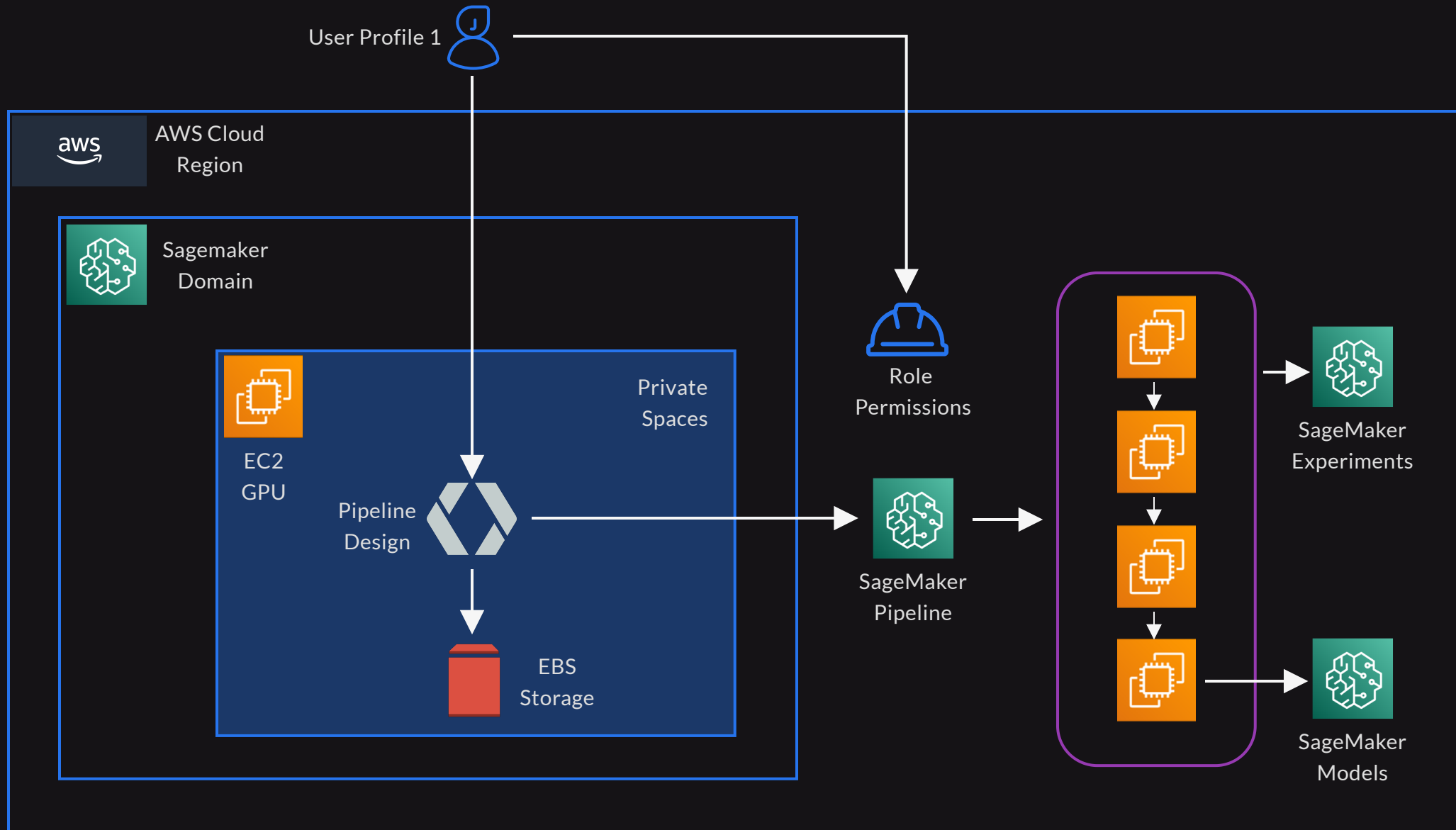
Callback

Lambda



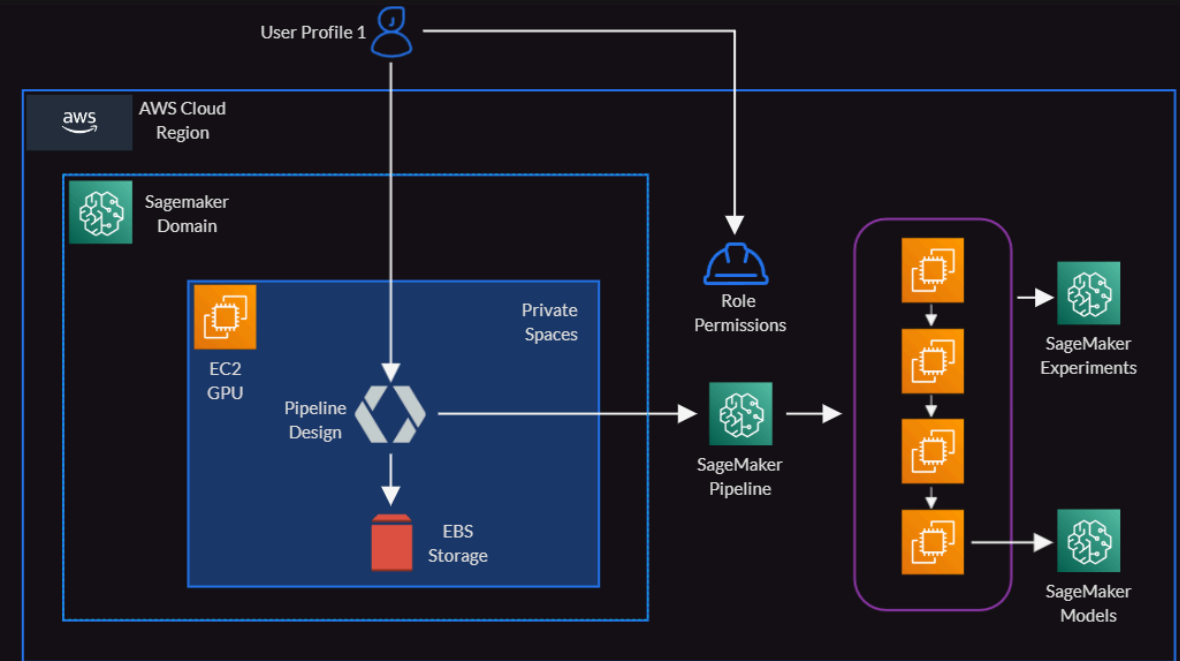
EventBridge

SageMaker Infrastructure



SageMaker Infrastructure

- Standardized library requirements using containers
- Modularized Data fetching and pre-processing
- Run different steps on different machines for cost saving
- Infra-scaling for pre-processing and hyperparameter tuning
- Automatic tracking in SageMaker Experiments
- Easy pipeline rerun using global parameters



Thank You
